🔹 Java Basic Syntax (C++ → Java Friendly)

1.Basic Program Structure

C++

```
#include<iostream>
using namespace std;

int main()
{
    cout<<"Hello";
    return 0;
}
```

Java

```
class Main
{
    public static void main(String[] args)
    {
        System.out.println("Hello");
    }
}
```

📌 **Key differences**

- No `#include`
- Everything inside a **class**
- `main` is `public static void`
- `System.out.println()` instead of `cout`

2.Data Types (Very Similar)

C++	Java
int	int
float	float
double	double
char	char
bool	boolean
string	String

```
int a=10;
float b=2.5f;
double c=3.14;
```

```
char ch='A';
boolean ok=true;
String name="Hisham";
```

⚠️ `String` starts with **capital S**

---

3. Taking Input (Single & Multiple)

Java uses `Scanner`

```
import java.util.Scanner;

Scanner sc=new Scanner(System.in);
```

Single input

```
int x=sc.nextInt();
double y=sc.nextDouble();
String s=sc.next();
```

Multiple inputs

```
int a=sc.nextInt();
int b=sc.nextInt();
int c=sc.nextInt();
```

Full line input

```
String line=sc.nextLine();
```

⚠️ `cin` ❌ → `Scanner` ✅

---

4. Output Printing

```
System.out.print("Hello");
System.out.println("World");
```

With variables

```
int x=5;
System.out.println("Value is "+x);
```

(No << chaining like C++)

5.If – Else

```
if(a>10)
{
    System.out.println("Big");
}
else if(a==10)
{
    System.out.println("Equal");
}
else
{
    System.out.println("Small");
}
```

✓ Same logic as C++

---

6. For Loop

C++

```
for(int i=0;i<n;i++)
```

Java

```
for(int i=0;i<n;i++)
{
    System.out.println(i);
}
```

✅ Identical syntax

---

7. While Loop

```
int i=0;
while(i<5)
{
    System.out.println(i);
    i++;
}
```

---

8. Do-While Loop

```
int i=0;
do
{
    System.out.println(i);
```

```
    i++;
}
while(i<5);
```

---

9. Taking Input Using Loop

```
Scanner sc=new Scanner(System.in);
int n=sc.nextInt();

int[] arr=new int[n];

for(int i=0;i<n;i++)
{
    arr[i]=sc.nextInt();
}
```

---

10. Output Using Loop

```
for(int i=0;i<n;i++)
{
    System.out.println(arr[i]);
}
```

---

11. Arrays

Declaration

```
int[] arr=new int[5];
```

Initialization

```
int[] arr={1,2,3,4,5};
```

2D Array

```
int[][] mat=new int[3][3];
```

---

12. Functions (Methods)

C++

```
int add(int a,int b)
```

Java

```
static int add(int a,int b)
{
```

```
    return a+b;
}
```

📌 `static` is important inside `main` class

---

13. Class & Object

```
class Student
{
    int id;
    String name;
}

public class Main
{
    public static void main(String[] args)
    {
        Student s=new Student();
        s.id=1;
        s.name="Hisham";
    }
}
```

✓ No `struct` in Java
✓ Everything is class-based

---

14. Constructor

```
class Student
{
    int id;
    Student(int x)
    {
        id=x;
    }
}
```

---

15. Inheritance (VERY IMPORTANT)

Single Inheritance

```
class A
{
    int x=10;
}

class B extends A
{
    int y=20;
```

```
}
```

📌 `extends` instead of `:`

---

16. Method Overriding

```
class A
{
    void show()
    {
        System.out.println("A");
    }
}

class B extends A
{
    void show()
    {
        System.out.println("B");
    }
}
```

---

17. Super Keyword

```
class B extends A
{
    void show()
    {
        super.show();
    }
}
```

---

18. Multiple Inheritance (Using Interface)

```
interface A
{
    void show();
}

interface B
{
    void display();
}

class C implements A,B
{
    public void show(){}
    public void display(){}
}
```

🚫 Java does NOT allow multiple inheritance using classes

19.Access Modifiers

Modifier	Meaning
public	Everywhere
private	Same class only
protected	Same package + child
default	Same package

---

20. Final Keyword

```
final int x=10;
```

❌ Cannot be changed (like `const`)

---

21. Exception Handling (Basic)

```
try
{
    int x=10/0;
}
catch(Exception e)
{
    System.out.println("Error");
}
```

🔁 C++ → Java Quick Mapping

C++	Java
cout	System.out.print
cin	Scanner
struct	class
bool	boolean
string	String
: (inheritance)	extends
header files	import